Checker Rewrite Triggered (Votes ≥ τ) → Re-synthesize Oracle & Rerun

OFFLINE: INVARIANT & INSTRUMENTATION

1 Invariant Generation

Segmented CoT + Historical-Bug RAG

Defense Specs & Compiler Semantics
Stack-Canary, SafeStack, PAC, CET/IBT, CFI
Target ABI, register conventions, prologue/epilogue

Historical CVE & Patch Corpus (RAG)
Homologue root-cause & silent-fix retrieval
CoT candidate extraction & ground-check

Security Invariant Catalog Φ
ϕ_1: Canary | ϕ_2: Layout | ϕ_3: Return

2 Initial Checker Construction

Deterministic Oracles (Static & Dynamic per ISA)

Static Checker
- AST & GIMPLE audit
- LLVM-IR / Asm scan

Pre-compile structure

Dynamic Checker
- Executable probe binary
- Stack/register overflow

Runtime verdict oracle

Multi-ISA Architecture Adaptation:
x86-64 | AArch64 | RISC-V

Four-State Verdict + Deterministic Evidence
PASS | FAIL (Bug) | N/A | ERROR

3 Source Code Mapping

3-Level Invariant Mapping via Free Agent + MCP

Free Coding Agent (e.g., Claude Code)
MCP Tools: call-graph | read/grep/ast

Level 1: Compiler Pass
pass_stack_protect / StackProtectorPass

Level 2: Target Functions
expand_used_vars() / LayoutStackNodes()

Level 3: Structs & Critical Conditions
has_vla, stack_protect_decl, canary_loc

4 Log Instrumentation & Build

Unique-ID Prefix Probing & Recompilation

gcc/cfgexpand.cc (Instrumented)

```
void expand_stack_protection() {
DEFUZZ_LOG("SP#a3f2", has_vla, off);
if (has_vla || has_vla_ptr) {
DEFUZZ_LOG("SP#a3f2:hit", canary_slot);
} emit_canary_prologue(); }
```

Instrumented Compiler IC
Rebuilt with deterministic log hooks
Unique UID Tagging · Trace Emitted

Bind Invariant ϕ_i + Checker Oracle

Deploy Instrumented Compiler IC

ONLINE · AGENTIC CLOSED-LOOP FUZZING

ORCHESTRATOR Lock Target Invariant ϕ_i + Checker Suite | Equal Time Budget T per ϕ | Sequential invariant campaign · Systematic coverage across defense matrix | Continuous Campaign Queue

5 Generator Agent

Checker-Aware C/C++ Seed Synthesis

LLM Seed Generation Agent
- Perceives defense invariant ϕ & checker logic
- Synthesizes complex C/C++ syntactic bounds

seed_sp_probe.c

```
// Targeted to provoke canary omission
void test(int len, char *src) {
char vla[len]; volatile int tag;
memcpy(vla, src, len); // trigger
sink(vla, &tag); }
```

Generation Objectives:
1. Satisfy compiler frontend syntax & types
2. Bypass middle-end dead-code elimination (DCE)
3. **Exercise critical struct & branch conditions**
4. **On FAIL → Lateral reasoning for more bugs**

Seed

Build & Dual Verification

Instrumented IC + Checker Oracle Evaluation

Compile with IC (GCC / LLVM)
Emits assembly, IR & unique UID log trace
Emits Diagnostic Logs: SP#a3f2:hit

Checker Oracle Evaluation

Static Audit
- Asm stack frame
- Canary location
- CFI jump tables

Dynamic Probe
- Runtime execution
- Exploit trigger
- Crash / Canary check

Structured Oracle Verdict + Telemetry
PASS / N/A — Invariant Holds | ERROR / WARN — Compile/Syntax Fail | FAIL (Bug) — Violation Found

Invariant Violated? (FAIL Verdict)

NO: PASS · N/A · ERROR (Send Telemetry to Feedback)

YES: FAIL

CONFIRMED BUG
Zero-Day Vulnerability
- C-Reduce automated PoC
- Human developer review

Feed to Historical CVE RAG Corpus →

Lateral Exploration
Synthesize more violation poses

Lateral Reasoning Loop: Burst diverse seed variants to find more violations for ϕ_i

6 Feedback Agent

Diagnostics-Aware Routing & Adaptation

Compiler & Checker Signals (MCP)
- Compiler warnings & AST diagnostic notes
- Checker static assertions & execution trace
- **Critical condition hit counter: hit_count(UID)**

Feedback Decision & Action

Decision 1: Guide Next Seed Round
- Condition not hit → Adjust seed data types & sizes
- Silent optimization (DCE) → Add volatile sinks

Steers Generator towards unexercised branches

Decision 2: Checker Rewrite (Threshold τ)
- Checker over-constrained or false-positive prone
- **Accumulated rewrite votes ≥ τ (e.g., 3 rounds)**

→ Triggers Phase 1 Checker Refinement & Rerun

Guidance Loop: Diagnostics-Guided Seed Mutation Hints